

Introduction

Rope is an AI-powered educational robot designed to interact with students through natural voice conversation, computer vision, and physical movement. It runs on a Raspberry Pi 5 with an ESP32 motor controller, using cloud-based speech recognition and a large language model to understand student questions and respond verbally while performing expressive gestures through servo-driven head and arm movements.

1. Project Overview

1.1 Project Goals

The primary goal of Rope is to create an interactive learning companion that helps students understand lessons through spoken dialogue. The robot listens to the student, processes their question through a voice pipeline backed by a cloud LLM, and responds with spoken answers while optionally moving its head, arms, or driving its wheels. A secondary goal is academic mode, where a mobile application can push lesson content to the robot so that it answers questions specifically about the current lesson material.

1.2 Target Users

The robot is designed for school and university students, primarily in Arabic-speaking and English-speaking educational environments. The default session language is Arabic, and the voice interface supports seamless switching between Arabic and English. The system is used in classroom settings where a robot sits on a desk and students interact with it by speaking.

1.3 Development Platform

The primary deployment platform is the Raspberry Pi 5 running Raspberry Pi OS, with the ESP32 DevKit handling motor and servo control over UART at 115200 baud. A secondary development and debugging platform is Windows desktop, where the vision and voice pipelines can be tested without the hardware layer. The `setup.sh` script handles Raspberry Pi provisioning including enabling the camera, UART interface, GPU memory split, and dialout group membership for serial access.

2. System Architecture

2.1 High-Level Architecture

The system is organized into four major layers. The hardware layer includes an ESP32 microcontroller connected to the Raspberry Pi via UART, driving an L298N dual H-bridge motor controller and three servo motors for head, right arm, and left arm. The hardware layer also includes a battery voltage divider connected to ESP32 GPIO 34. The perception layer consists of a camera module feeding a multi-stage vision pipeline and a USB microphone feeding a voice activity detection and speech recognition pipeline. The intelligence layer is built around an LLM integration module that communicates with the OpenRouter API, supported by a SQLite-backed session manager with sliding window memory and automatic summarization. The presentation layer comprises a pygame-based animated face rendered on the HDMI display and a text-to-speech module using Microsoft Edge TTS for audio output.

2.2 Data Flow

Voice interaction begins when the student speaks into the microphone. The voice pipeline captures audio through the `sounddevice` library at the device native sample rate, resamples it to 16000 Hz using `scipy.signal.resample_poly`, and passes 32-millisecond chunks to the Silero VAD model running in PyTorch. When speech is detected, the pipeline accumulates audio chunks into a segment. After a configurable silence timeout defaulting to 800 milliseconds, the segment is enqueued for the worker thread, which sends it to Google Speech Recognition for transcription. The

transcribed text is checked against a wake word list using fuzzy matching with a 70 percent threshold. If a wake word is detected, the robot responds with a greeting and activates listening mode. The next utterance is then checked for direct motor commands such as forward or stop. The text is then sent to the LLM along with the current vision context and academic context if active. The LLM response is played through the TTS engine while the robot's face displays the speaking animation.

On the vision side, the camera captures frames which are processed by up to six modules depending on the configured profile. Face tracking uses OpenCV DNN with a Caffe SSD model. Gesture detection uses contour analysis with HSV and YCrCb skin masking and convexity defect finger counting. Object detection uses YOLOv8s with lazy loading. Scene segmentation uses YOLOv8s-seg. Obstacle detection uses Lucas-Kanade optical flow to compute divergence and expansion ratio. Emotion detection uses a PyTorch CNN with seven classes but is currently hardcoded disabled. The results are merged into a shared context dictionary and the best frame from a 10-frame buffer is selected and passed to the LLM when real data is detected.

3. Software Components

3.1 Voice Pipeline

The voice pipeline in `voice/pipeline.py` runs two threads. The stream thread captures audio using `sounddevice.InputStream` with a callback that processes each chunk. The chunk is first resampled if the device sample rate differs from the target 16000 Hz. Resampling uses polyphase filtering via `scipy.signal.resample_poly` with a ratio computed from the GCD of the source and target rates. The resampled chunk is then passed to the VAD module. Speech segments are detected using a state machine that tracks a pre-roll buffer of 300 milliseconds, a minimum speech duration of 500 milliseconds, and a silence timeout of 800 milliseconds. When a segment ends, it is placed on a bounded queue with a maximum size of three segments.

The worker thread dequeues segments and transcribes them using the `speech_recognition` library with the Google Speech API. The ASR module in `voice/asr.py` supports auto mode, where it tries Arabic language code `ar-EG` first then English `en-US`. The first successful recognition determines the detected language. If the transcribed text is a wake word, the robot plays a greeting and sets an active flag. Subsequent speech is checked against a hardcoded dictionary of motor command keywords that includes Arabic and English phrases. For each recognized keyword, the corresponding motor method is called with a default duration. The text is then sent to the LLM via the chat method, and the response is played through the TTS module.

The VAD module in `voice/vad.py` wraps the Silero VAD model loaded via `torch.hub` from the `snakers4/silero-vad` repository. The model can also be loaded from a local path specified by `ROBOT_VAD_MODEL_LOCAL_PATH`, or from a fallback directory at `config/snakers4-silero-vad`. The decision threshold defaults to 0.60 and can be changed at runtime through the `set_threshold` function. Audio preprocessing converts stereo to mono by averaging channels, clips amplitude to a configurable maximum, and resamples linearly if needed.

The TTS module in `voice/tts.py` uses `edge-tts` to generate MP3 audio files, then plays them using `pygame.mixer.music`. It supports two voices: `ar-SA-HamedNeural` for Arabic and `en-US-GuyNeural` for English. Playback runs in a daemon thread and can be interrupted by calling `stop`, which sets a threading event and unloads the `pygame` mixer. The module fires callbacks on playback start and finish, which the voice pipeline uses to update the face to `SPEAKING` and `IDLE` states respectively.

The filler phrase system plays a random phrase from a predefined list in the detected language before the LLM responds. Arabic filler phrases include single words for thinking and English phrases include short acknowledgments. If the LLM is unavailable after three retries, a random fallback message apologizing for the connection issue is played instead.

3.2 Vision Pipeline

The vision subsystem in `vision/pipeline.py` creates a separate thread that continuously captures frames from the Raspberry Pi Camera Module 3 using `picamera2`. Each frame is processed through a chain of modules. The camera module in `vision/camera.py` configures the camera at 640x480 resolution in RGB888 format with a frame rate limit of 5 FPS to prevent overloading the CPU. Auto-exposure is enabled with exposure compensation set to 1.3 in the tuning file.

The vision system supports three profiles. The MINIMAL profile enables only obstacle detection. The BALANCED profile enables obstacle and emotion detection. The FULL profile enables obstacle detection, emotion detection, object detection, and scene analysis but only on desktop systems. The profile is selected automatically based on the configured ROBOT_VISION_PROFILE environment variable.

Each module implements a vision_modules base interface providing an analyze method that takes a raw frame and a context dictionary and returns an updated context. The pipeline passes results through a shared context dictionary and selects the best frame for LLM submission from a 10-frame buffer. If no real objects are detected, the frame is not submitted. Frame submission is subject to a debounce period of one second and a maximum submission interval of 30 seconds. The pipeline also implements thermal throttling by checking CPU temperature via /sys/class/thermal/thermal_zone0/temp and skipping frames when the temperature exceeds 70 degrees Celsius. Adaptive load shedding under high temperature disables vision analysis for a cooldown period of 5 seconds.

3.3 Vision Modules

The face tracker module in vision/modules/face_tracker.py uses OpenCV DNN with the deploy.prototxt and res10_300x300_ssd_iter_140000.caffemodel files. It detects all faces in the frame, selects the largest one, and computes the bounding box width to determine camera-to-subject distance. It supports four distance categories: out of range beyond 250 pixels, very close with emotion analysis, ideal, and too far. The module returns face bounding box coordinates and the stability of the face region. It can optionally enable emotion analysis on the face region.

The gesture module in vision/modules/gesture.py implements hand detection through two masking methods applied sequentially on the lower third of the frame. It first applies an HSV mask with hue range 0 to 30 and saturation range 15 to 200 for skin color detection, then applies a YCrCb mask with Cr range 133 to 173 and Cb range 77 to 127. Contours are filtered by area with a minimum of 3000 square pixels. For each valid contour, convexity defects are computed and finger counting is performed by counting defects with a depth greater than a threshold. The total count is classified into gestures: fist for two or fewer fingers, open hand for four or more, and pointing for three fingers.

The object module in vision/modules/objects.py lazily loads the YOLOv8s model using the ultralytics library the first time analyze is called. It runs inference on the frame and filters detections by the default COCO dataset classes, a confidence threshold of 0.25, and an IOU threshold of 0.45. Only detections classified as person, book, bottle, cup, cell phone, laptop, or remote are returned. Each detection includes the class index, class name, confidence score, and bounding box coordinates.

The obstacle module in vision/modules/obstacle.py computes dense optical flow using the Farneback algorithm. The frame is resized to a quarter resolution and converted to grayscale for processing. Optical flow between consecutive frames is computed using OpenCV calcOpticalFlowFarneback with three pyramid layers, a window size of 15, and three iterations. The flow components are scaled back to original resolution and divergence is computed using Sobel derivatives. The expansion ratio is derived from the divergence image. An obstacle warning is triggered when the expansion ratio exceeds a threshold of 0.3.

The scene module uses the same YOLOv8s model from the objects module for scene segmentation. It applies the segmentation model to produce masks and classifies the environment as classroom, library, office, or outdoor based on detected objects. It also reports a scene change event when the set of visible classes changes between frames.

The emotion module uses a PyTorch CNN with seven output classes representing happy, sad, angry, surprised, fearful, disgusted, and neutral. The model is expected at config/emotion_model/emotion_model.pth. It preprocesses face regions by resizing to 48x48 pixels and converting to grayscale. However, the module is currently hardcoded to return neutral for all calls with a debug message indicating the emotion system is disabled.

3.4 LLM Integration

The LLM integration in llm/module.py provides the top-level Orchestrator class that coordinates the voice pipeline, vision pipeline, motor controller, and session persistence. It initializes all subsystems and passes them to the main loop. The main loop polls the voice pipeline for transcribed text and processes it.

The OpenRouterChat class in the same file manages LLM calls through the OpenRouter API. It constructs a session manager backed by SQLite for conversation persistence. The system prompt includes the robot personality as an Arabic-speaking Egyptian friend and teacher named Ropo who explains academic subjects simply and funly while answering only about the current lesson topic. It provides guidance on responding about topics ranging from science,

math, history, and languages to computer science, technology, and engineering.

The LLM call is configured with the DeepSeek V3 model with a temperature of 0.7, a maximum of 4096 output tokens, and a 90-second timeout. HTTP requests use the httpx library with retries: three attempts with exponential backoff starting at 2 multiplied by 20 seconds, using jitter. The session ID defaults to default and is overridden by the academic API to academic_api.

The session memory in llm/session.py maintains a SQLite database of sessions. Each session has a sliding window of the last N messages where N is configurable from 15 to 100 with a default of 50. When the window fills up, the oldest messages are evicted and a summarization pass is called. The summarization prompt asks the LLM to condense the conversation into a brief Arabic summary. However, the summary is stored in the database but is NOT loaded back into the context. The session storage also supports context injection from the academic API through the set_context method, which allows addition, replacement, or clearing of context sections by key.

3.5 Academic Mode

The academic API server in academic/server.py runs a FastAPI server on port 8001 with no authentication. It provides three endpoints. The GET status endpoint returns the current block state and a readiness flag. The POST context endpoint accepts a JSON body with student_id, subject, lesson_title, and lesson_content fields. It updates a global AcademicContext with the lesson material. The POST ask endpoint accepts a JSON body with student_id and question fields. It creates a temporary OpenRouterChat session, injects the academic context directly into the system prompt, asks the LLM the question, and returns the response without persisting the conversation to the session database.

The AcademicContext class in academic/context.py stores the current subject, lesson title, and lesson content. It wraps the lesson content as a system-level instruction that tells the LLM to answer based only on the provided content and refuse to answer questions outside the lesson scope.

The mobile device pushes lesson content before the student starts asking questions, but the current implementation does NOT provide PDF upload or file management endpoints. The mobile application must extract text from PDFs on the device side before sending it as plain text to the POST context endpoint.

3.6 Motor Controller

The hardware layer communicates with an ESP32 over a serial UART interface at 115200 baud. The motor_controller.py module on the Raspberry Pi side handles port auto-detection, connection, and command sending. It supports three strategies for finding the ESP32. First, it tries the port specified by ROBOT_MOTOR_PORT if configured. Second, on Linux it scans /dev/ttyAMA, /dev/ttyUSB, and /dev/ttyACM device paths. Third, on Windows it enumerates all available COM ports. For each candidate port, it opens a connection with a 2-second baud rate detection delay, sends a ping command, and waits for a response containing ROPE Motor Controller Ready or BAT:. The first port that responds successfully is selected.

The motor command protocol uses single uppercase characters. F moves forward for 500 milliseconds. B moves backward for 500 milliseconds. L turns left for 250 milliseconds. R turns right for 250 milliseconds. S stops all motors. The servo command format is SERVO_NAME:ANGLE where angle ranges from 0 to 180. HEAD controls the head pan servo, ARM_R controls the right arm servo, and ARM_L controls the left arm servo. The CENTER command simultaneously returns all servos to their neutral 90-degree positions. The HAPPY command triggers a pre-programmed animation on the ESP32. The connection maintains a read thread that continuously reads lines from the serial port and stores them in a buffer.

3.7 Battery Monitoring

The battery monitor in hardware/battery_monitor.py runs a background thread that reads the most recent battery voltage from the motor controller serial buffer. The ESP32 sends battery packets every 2 seconds in the format BAT:7.45. If no battery data is received within 30 seconds, the battery state is set to unknown. The voltage thresholds are read from environment variables. The low voltage threshold defaults to 7.1 volts. The critical voltage threshold defaults to 6.5 volts. When voltage drops below the low threshold, a warning is logged and the face switches to a battery warning state. After 30 seconds of continuous low voltage, the shutdown countdown begins. When voltage drops below the critical threshold, the robot immediately shuts down all subsystems and waits for recharging.

3.8 Face Display

The face module in `voice/face.py` renders a 480x320 pixel animated face on the HDMI display using pygame with the SDL hardware cursor hidden. It supports seven states: idle, listening, thinking, speaking, greeting, sleeping, and battery warning. Each state maps to visual animations. The theme system has three themes: default with dark background and light eye colors, dark with light background and dark eye colors, and minimal with simple rectangular features without mouth animation. Eye animations include blinking, scanning, and the emotion-specific visual response during the listening state. The face detects idle state through a threading event and enters a pre-sleep state after 90 seconds of idle time, showing a sleeping animation for 15 seconds before turning off the display.

3.9 Configuration System

All configurable parameters are managed through the `config/settings.py` module using Python dataclasses. The `get_settings` function reads environment variables from a `.env` file using `python-dotenv` and validates them against defined defaults and types. The main configuration classes include `SpeechSettings` for VAD threshold, silence timeout, language, and wake word sensitivity. `MotorSettings` for motor port and baud rate. `LLMSettings` for API key, model, temperature, timeout, vision enablement, session window size, and retry configuration. `AudioSettings` for chunk size, sample rate, device index, and pre-roll duration. `VisionSettings` for camera resolution, frame rate, profile, and enabled modules. `HardwareSettings` for battery voltage thresholds and GPIO pins. The settings are validated on startup and missing critical parameters such as the OpenRouter API key result in an error log but allow the system to continue in a degraded state where LLM calls will fail.

3.10 Shutdown System

The shutdown system consists of two components. A server runs in `shutdown/main.py` as a FastAPI application on port 8000 with a configurable token passed via the Authorization Bearer header. It provides a POST shutdown endpoint that sets a global shutdown flag and a GET status endpoint that returns the current shutdown state. The client in `robot_shutdown_client.py` connects to this server, authenticates with the token, and sends the shutdown command. The main application loop in `main.py` polls the shutdown flag at regular intervals and performs an orderly shutdown when it detects the flag is set. The shutdown system allows a mobile application to remotely power down the robot in a classroom environment.

4. Development and Deployment

4.1 Development Setup

The project requires Python 3.10 or higher. Dependencies are listed in `requirements.txt` and include torch for the VAD and emotion models, `opencv-contrib-python` for computer vision, `picamera2` for Raspberry Pi camera access, `sounddevice` and `PyAudio` for audio capture, `edge-tts` for speech synthesis, `pygame` for face rendering, `pyserial` for UART communication, `fastapi` and `uvicorn` for API servers, `httpx` for HTTP requests, `python-dotenv` for configuration, and `scipy` for audio resampling. On Raspberry Pi, the `setup.sh` script installs system dependencies and Python packages.

4.2 Testing

The project includes no automated test suites. All testing is performed manually by running the robot on the target hardware and observing behavior. The system logs extensive debug output at each pipeline stage to aid in manual verification and troubleshooting.

4.3 Deployment

Deployment requires a Raspberry Pi 5 with the Camera Module 3 connected and a USB microphone. The ESP32 must be flashed with the firmware in `hardware/esp32/main.cpp` using the Arduino IDE or PlatformIO with the Arduino framework and the ESP32 Servo library. The `.env` file must be populated with at minimum the OpenRouter API key. The robot is started by running `main.py` with appropriate environment variables set.

5. Conclusion

Rope is a well-structured educational robot project that integrates modern AI capabilities including speech recognition, LLM-based conversation, computer vision, and physical motor control into a single embedded system. The architecture cleanly separates concerns across voice, vision, intelligence, hardware, and presentation layers. The module-based design allows each subsystem to be developed and tested independently. The system demonstrates a practical application of cloud-based AI on edge hardware, using the Raspberry Pi 5 as an orchestration platform that bridges cloud intelligence with physical interaction. Future development areas include expanding the vision emotion module, implementing conversation summary injection into LLM context, adding PDF upload support for the academic mode, and building a mobile application companion for remote lesson management and robot monitoring.